

Airbus A350-900 | Abstraktion

Kategorie	Beschreibung
Definition von Abstraktion	Abstraktion ist das Prinzip der objektorientierten Programmierung, bei dem nur die wesentlichen Merkmale eines Objekts dargestellt werden, während unwichtige Details verborgen bleiben. Sie ermöglicht die Modellierung komplexer Systeme auf einer höheren, verständlicheren Ebene.
Anwendungsfall (Airbus A350-900)	Bei der Modellierung eines Airbus A350-900 werden komplexe Systeme wie Engine , FlightControlSystem und Avionics abstrahiert, um die Interaktionen auf einer höheren Ebene darzustellen, die für die Steuerung und Verwaltung des Flugzeugs relevant sind.
Abstraktionsebenen	Das Modell des Airbus A350-900 kann in mehrere abstrakte Ebenen unterteilt werden, um verschiedene Teile des Flugzeugs zu isolieren und damit den Zugriff auf relevante Details zu vereinfachen.
Höchste Abstraktionsebene	Auf dieser Ebene wird der Airbus A350-900 als gesamtes Objekt betrachtet, mit Eigenschaften wie Flugzeugkennung, Flugstatus, Reichweite und aktuellen Betriebsparametern, ohne auf die Details einzelner Systeme einzugehen.
Mittlere Abstraktionsebene	Das Flugzeug wird in wesentliche Teilsysteme unterteilt. Jedes System ist ein abstraktes Modell eines komplexeren, realen Systems, wie zum Beispiel: FlightControlSystem , Engine , Avionics , LandingGear , Cabin , FuelSystem .
Niedrigste Abstraktionsebene	Auf dieser Ebene werden spezifische Komponenten innerhalb eines Systems abstrahiert. Zum Beispiel besteht das Engine -System aus vielen Komponenten wie Compressor , Turbine , FuelInjector , ThrustControl , die jeweils über eigene Eigenschaften und Methoden verfügen.
Abstraktion durch Schnittstellen	Schnittstellen definieren das Verhalten und die Interaktion zwischen verschiedenen Komponenten, ohne Details zur Implementierung zu offenbaren. Ein ControlSystemInterface könnte Methoden wie <code>adjustAltitude()</code> , <code>adjustSpeed()</code> , <code>correctCourse()</code> bereitstellen, während das spezifische FlyByWireSystem diese Methoden im Detail umsetzt.
Abstraktion durch abstrakte Klassen	Abstrakte Klassen bieten eine Basisklasse mit gemeinsamen Eigenschaften und Methoden für verwandte Objekte. Zum Beispiel könnte die abstrakte Klasse AircraftSystem grundlegende Funktionen wie <code>start()</code> und <code>shutdown()</code> definieren, während die spezifischen Systeme wie Engine und Avionics diese Methoden implementieren.

Kategorie	Beschreibung
Beispiel: Triebwerksabstraktion	Anstatt jedes Engine -Modell einzeln zu implementieren, könnte eine abstrakte Klasse Engine Methoden wie <code>start()</code> , <code>stop()</code> , <code>increaseThrust()</code> enthalten, während spezifische Triebwerke wie RollsRoyceTrentXWB diese Methoden mit ihren eigenen Parametern und Details erweitern.
Abstraktion im Flugsteuerungssystem	Ein allgemeines FlightControlSystem könnte Methoden wie <code>adjustAltitude()</code> oder <code>changeDirection()</code> abstrahieren, ohne sich um die spezifische Implementierung von FlyByWire oder HydraulicSystems zu kümmern. Diese Systeme würden dann die allgemeinen Methoden implementieren, je nach ihrer Funktionsweise.
Abstraktion im Treibstoffsystem	Das FuelSystem könnte in einer abstrakten Klasse grundlegende Funktionen wie <code>checkFuelLevel()</code> , <code>fuelConsumption()</code> und <code>refuel()</code> definieren, während spezielle FuelPump -Komponenten diese auf ihre eigene Weise realisieren.
Vorteile der Abstraktion	Komplexitätsreduktion: Nur relevante Details werden gezeigt, wodurch die Komplexität des gesamten Systems verringert wird. Wartbarkeit: Änderungen an spezifischen Implementierungen wirken sich nicht direkt auf die gesamte Systemstruktur aus. Wiederverwendbarkeit: Abstrakte Modelle können leicht in verschiedenen Systemen wiederverwendet werden. Erweiterbarkeit: Neue Komponenten oder Systeme können ohne großen Einfluss auf das bestehende System hinzugefügt werden.
Zusammenhang mit anderen OOP-Prinzipien	Kapselung (Encapsulation): Abstraktion verbirgt die Implementierungsdetails und schützt so vor direktem Zugriff. Vererbung (Inheritance): Abstraktion nutzt Vererbung, um spezifische Implementierungen von Basisklassen zu erben und zu erweitern. Polymorphie (Polymorphism): Verschiedene konkrete Implementierungen von abstrakten Klassen oder Interfaces können austauschbar verwendet werden, je nachdem, welche konkreten Objekte zum Laufzeitzeitpunkt erstellt werden.